을숙도 아키텍쳐 쏘아보기

을숙도 아키텍쳐는 프로세서에서
여러 명령을 동시에 처리하는 **슈퍼스칼라**와
순서와 상관없이 실행할 수 있는 명령을 즉시 처리 하는 **비순차적 실행 처리**
가 적용된 구조를 구현합니다.

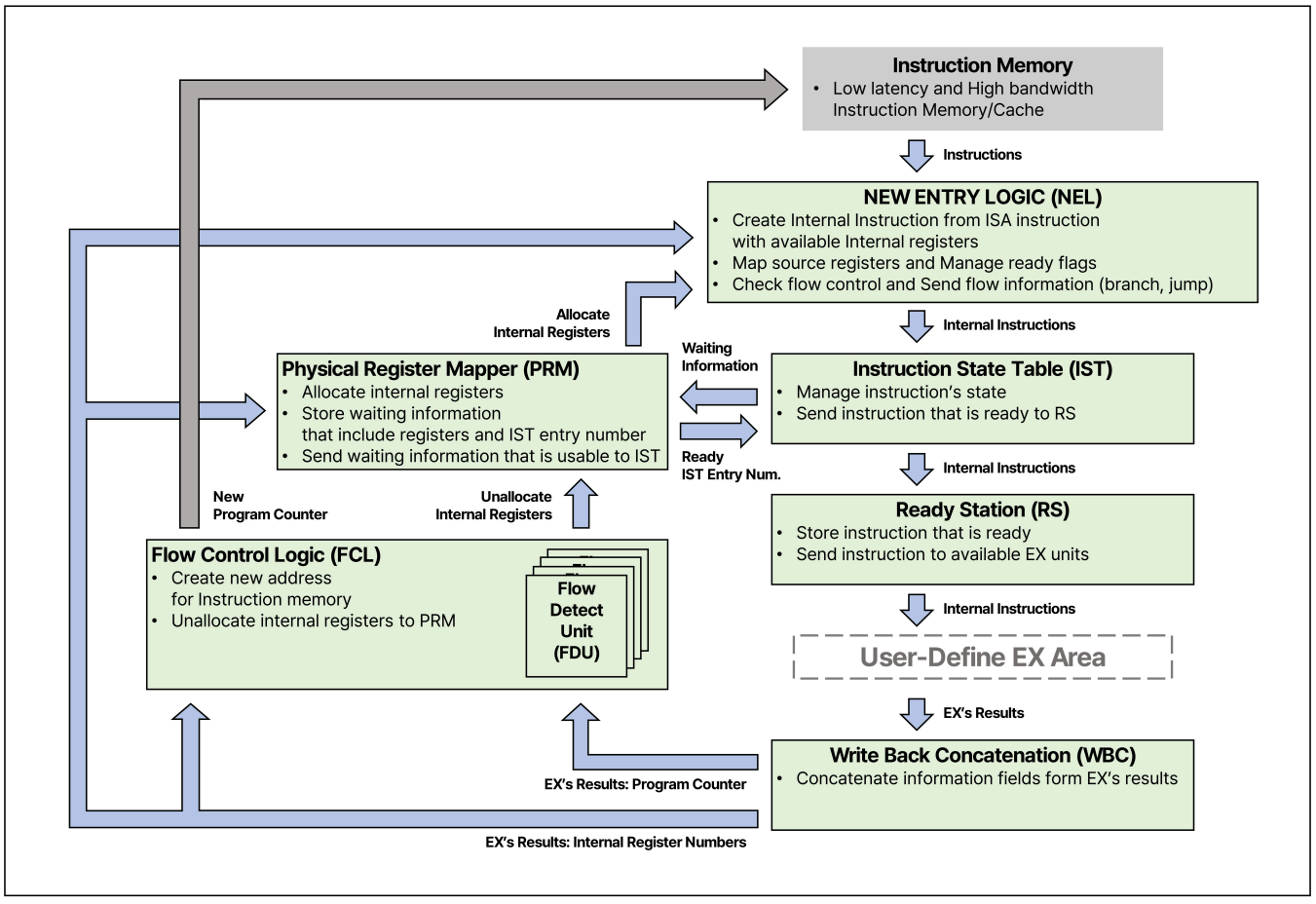
전체적인 컨셉은요?

프로세서는 명령어를 처리하여 특정한 동작을 진행하는 디지털 회로입니다.
을숙도 아키텍쳐의 중점은 **명령의 대기, 가용 상태를 확인하고 관리**하는 것입니다.
이때 데이터 종속성 문제를 해결하기 위해 **레지스터 체계를 바꾸어 다른 이름으로 변경**합니다.
특히 새롭게 입력되는 명령의 결과 레지스터는 **항상 새로운 레지스터 이름으로 변경**하여
같은 이름을 가진 레지스터를 수정할 때 발생하는 WAW 해저드 문제를 해결하고자 합니다.

어떤 부분을 중점으로 구현하고 있나요?

실질적인 구현을 위해 **내부적인 명령/레지스터 번호를 할당하고 반환하는 구조**를 가집니다.
명령의 대기, 가용 상태를 확인하고 관리하기 위해,
명령에서 레지스터 번호를 항상 감시하는 구조를 가지는것이 일반적입니다.
특히 상용 프로세서에서는 *Contents Addressable Memory(CAM)*이라는 구조를 사용하여
명령이 저장된 각각의 엔트리에서 필요한 소스 레지스터 만큼 비교기를 사용하고 연결하는 구조를 가져
Glitch와 Fan-Out 관점에서 불리합니다.
이런 문제를 완화하고자,
(내부의 체계로 변환된)레지스터가 필요한 *명령의 번호*를 **레지스터 번호 기준으로 저장**하여
실행 유닛에서 결과를 만들어낸 레지스터 번호를 전달받아
저장된 명령의 번호로 명령의 상태를 변경하는 구조로 설계하고 있으며,
이에 따라 CAM에 비해 상당히 감소된 Fan-Out과 비교기 수를 도출하여
높은 전력 효율을 기대하고 있습니다.

컨셉에 따라 어떻게 동작하나요?



을숙도 아키텍쳐는 **명령의 대기, 가용 상태를 확인하고 관리**하기 위해
프로세서의 스케줄러를 6개의 구획으로 나누었습니다.
명령의 입력부터 대기, 가용 상태로 변경과 최종 처리 순서를 반영하여 설명하면,

1. 새로운 명령을 입력받아, 이때 명령의 레지스터들을 내부 체계의 이름으로 변경하며,
   새로운 명령에서 필요한 레지스터의 준비여부를 모아 내부에서 처리하는 명령 체계로 변환하는
   New Entry Logic(NEL)
2. 내부 체계로 변경된 명령을 저장하고, 필요한 레지스터들의 준비 여부를 업데이트 하며,
   명령의 소스 레지스터들이 모두 준비되면 해당 명령을 외부로 출력하는
   Instruction State Table(IST)
3. 내부 체계로 변경된 레지스터 이름을 기준으로,
   특정 레지스터가 사용되는 명령의 내부적인 번호를 저장해두고,
   실행 유닛에서 처리가 완료된 레지스터 번호를 받아 해당 레지스터의 명령 번호를 출력하는
   Physical Register Mapper(PRM)
4. 준비된 명령들이 필요한 실행 유닛에 들어가도록 배치하고,
   각 실행 유닛이 동작할 수 있는 상태에 전달할 수 있도록 버퍼링하는
   Ready Station(RS)
5. 실행 유닛에서 완료된 명령의 레지스터 번호를 전달하고,
   명령의 순서를 바꾸는 정보를 전달하거나 외부 I/O로 데이터를 교환하는
   Write Back Concatenation(WBC)
6. 명령의 순서를 전달하고, 분기/점프로 둘러싸인 Basic Block 단위로 순서를 관리하며,
   명령의 처리와 함께 새로운 레지스터로 할당되면서 필요없는 레지스터를 반환하는
   Flow Control Logic(FCL)

("내부적인"과 같은 내용을 서술하지 않은 경우 외부에서 들어온 정보 기준입니다.
명령:PC / 명령의 내부적인 번호:IST 엔트리 번호)

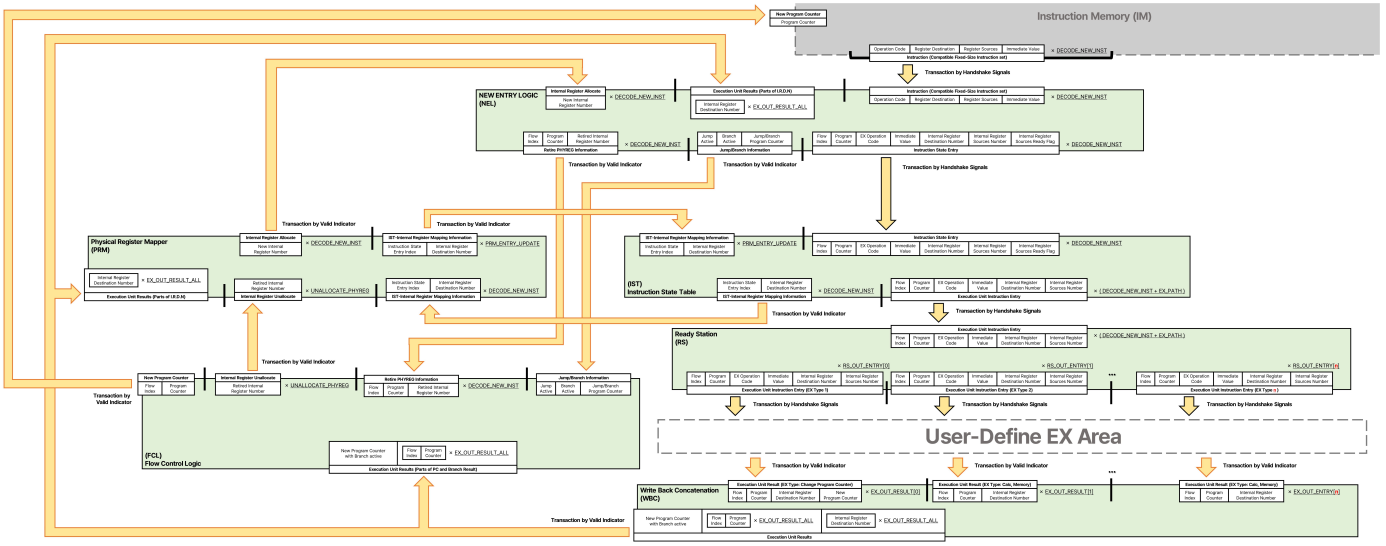
전체적으로 할당 가능한 내부 레지스터 번호는 1번부터 시작하며,
내부 레지스터 0은 항상 "0" 값을 출력하는 구조로 만들어져 있습니다.
따라서, 내부 레지스터 0은 항상 준비되어 있고, 수정할 수 없는 속성을 가집니다.

이 전체 모듈은 어떤 입력을 받고 어떤 출력을 하나요?

필수적인 I/O에 대해서 새로운 명령을 입력 받고,
출력으로는 필요한 명령의 주소를 전달합니다.

해당 프로세서의 성격에 따라, 메모리 접근 I/O 구성과 특정한 신호들을 외부와 교환할 수 있습니다.

내부적으로 모듈들은 어떻게 연결되나요?



각 모듈은 특정한 데이터 구조들을 **Valid 발생 단위**와 **Handshake 기반 전송 단위**로 전달합니다.

데이터 수신에서,
Valid 발생 단위의 데이터 전송에서 수신부는 항상 받을 준비가 되어 있어야 합니다.
Handshake 기반 전송 단위에서 **Ready 필드**를 통해 결정합니다.
데이터 송신에서, Valid 발생 단위의 데이터 전송과 Handshake 기반 전송 단위에서 모두
유효한 데이터 위치의 Valid 필드를 통해 결정합니다.

Handshake 기반 전송 단위의 구체적인 설명

신호는 다음과 같습니다.

Signal Name	Direction	Description
Valid	Sender->Recevier	데이터의 유효성
Data	Sender->Recevier	전송되는 데이터
Get	Recevier->Sender	가져가기 신호

을숙도 구조에서 Handshake 데이터 전달은 가져가기 신호를 가능한 항상 띄웁니다. Valid와 Get 양쪽의 동일한 비트 위치의 필드가 발생하면 **데이터 전송완료**입니다.

간단한 예시를 보이면
(가져기기 신호가 항상 발생중인 경우)

1. 수신하는 부분에서 Get 신호는 내부 모듈이 수신 불가능한 경우를 제외하고 항상 발생합니다.
2. 이때 전송하는 부분에서 데이터 전송이 필요하다면 Data를 출력하고, 동시에 Valid 신호를 전달합니다.
3. 수신하는 부분은 Valid 신호가 발생된 것 보고 처리합니다.
4. 다음 클럭 사이클에 전송하는 부분이 데이터 전송 준비가 안된경우, Valid 신호는 비활성화 합니다.

(가져가기 신호가 발생하지 못하는 경우)

1. 수신하는 부분에서 내부 모듈이 수신 불가능한 경우라 Get 신호가 발생하지 않습니다.
2. 이때 전송하는 부분에서 데이터 전송이 필요하다면 Get 여부와 상관 없이 Data를 출력하고, 동시에 Valid 신호를 전달합니다.
3. 수신하는 부분은 수신 가능한 경우에 바로 Get 신호를 발생하고, 전송하는 부분은 Get 신호가 발생될 때까지 대기합니다.
4. Get이 발생하면 수신하는 부분은 Valid 신호가 발생된 것 보고 처리합니다.
5. 다음 클럭 사이클에 전송하는 부분이 데이터 전송 준비가 안된경우, Valid 신호는 비활성화 합니다.

Valid 기반 전송 단위의 구체적인 설명

Signal Name	Direction	Description
Valid	Sender->Recevier	데이터의 유효성
Data	Sender->Recevier	전송되는 데이터

Valid 신호가 발생하면 해당 데이터를 항상 수신해야 합니다.

간단한 예시를 보이면

1. 전송하는 부분에서 데이터 전송이 필요하다면 Data와 Valid 신호를 전달합니다.
2. 수신하는 부분은 데이터를 수신받고 처리합니다.
3. 다음 클럭 사이클에 전송하는 부분이 데이터 전송 준비가 안된경우, Valid 신호는 비활성화 합니다.

EX 연결을 위한 규칙

을숙도 아키텍쳐는 Execution Unit을 모두 커스텀 하고 추가하는 구조입니다.
이때 사용되는 EX의 명령 포맷과 결과 포맷은 특정한 구조를 가져야 합니다.
반드시, 모든 커스텀 EX 모듈들의 입력 명령 포맷은 동일해야 합니다. 결과 포맷은 점프/분기 및 상태제어 EX를 제외하면 동일한 구조를 가집니다.
LSB부터 MSB로 왼쪽부터 오른쪽으로 입력 명령 포맷을 나타내면

Program Counter	Flow Index	Micro-Opcode	Immediate Value	RD Address	...RS(1~n) Addresses List...	...RS(1~n) Values List...

형태로 배치됩니다.
각 필드의 내용은

- **Program Counter**: Instruction Memory에서 해당 명령의 주소입니다.
- **Flow Index**: 해당 명령의 영령 윈도우 번호입니다. 점프/분기로 명령 윈도우가 구분되는데, 이 단위를 Flow Index로 구분됩니다.
- **Micro-Opcode**: Execution Unit의 Opcode(명령 구분자)입니다.
- **Immediate Value**: 명령에 포함된 즉시 입력되는 값입니다.
- **RD Address**: 결과가 저장되는 레지스터 목적지 번호입니다.
- **RS(1~n) Addresses List**: 필요한 레지스터 소스 번호입니다.
- **RS(1~n) Values List**: 필요한 레지스터 소스의 값입니다.

LSB부터 MSB로 왼쪽부터 오른쪽으로 일반적인 EX의 출력 결과 포맷을 나타내면

Program Counter	Flow Index	RD Address	RD Value

형태 이고, Branch EX의 출력 결과 포맷을 나타내면

Program Counter	Flow Index	RD Address	RD Value	Branch Active	New Program Counter

형태가 됩니다.

파라미터

을숙도 아키텍쳐는 확장 가능한 구조를 가집니다.
이에 따라, 파라미터를 이용하여 확장 가능한 RTL 코드를 제공~~하고 있습니다.~~ 할 예정입니다.

동일한 데이터 구조를 복수로 전달하기 위해 파라미터를 사용합니다.

파라미터의 이름 구조는

- `IS_INST_*`: Instruction Set에 따른 파라미터의 접두사 입니다.
- `EX_INST_*`: Execution Unit에 입력되는 명령의 정보에 대한 파라미터의 접두사 입니다.
  반드시, 모든 커스텀 EX 모듈들의 입출력 정보 파라미터는 공유되어야 합니다.
- `STRUCT_*`: 구조적인 설정을 위해 사용되는 파라미터의 접두사 입니다.

Parameter of Instruction Set

Instruction Set에 대한 파라미터입니다. Instruction Set의 명령 포맷 정보를 입력합니다.

- `IS_INST_PC_BITWIDTH`: Instruction Set에 정의된 명령 메모리의 주소 길이입니다.
- `IS_INST_PC_STEP`: Instruction Set에 정의된 명령의 주소에서 구분되는 단위입니다.
- `IS_INST_BITWIDTH`: Instruction Set에 정의된 명령의 길이입니다.
- `IS_INST_REGS`: Instruction Set에 정의된 범용 레지스터 갯수입니다. CSR과 같은 전용 레지스터는 MicroOP을 이용합니다.
- `IS_INST_OPERANDS`: Instruction Set에 정의된 명령의 소스 레지스터 수입니다.
- `IS_INST_IMM`: Instruction Set에 정의된 Immediate 값의 너비입니다.

Parameter of Execution Unit

Execution Unit에 대한 파라미터입니다. 상단 포맷에 맞게 EX의 명령 포맷 정보를 입력합니다.

- `EX_INST_MICROOP_BITWIDTH`: Execution Unit용 명령의 비트 너비입니다.

Parameter of EULSUKDO Structure

을숙도 아키텍쳐의 구조에 대한 파라미터입니다. 원하는 구조에 대한 정보를 입력합니다.

- `STRUCT_DECODE_NEW_INST`: 한 사이클에 Instruction Memory로 부터 입력되는 명령어의 최대 개수이며, 한번에 디코딩 되는 단위와 동일합니다.
- `STRUCT_INST_STATE_ENTRIES`: Instruction State Entry 갯수입니다.
- `STRUCT_PHYREGS`: 내부 레지스터 갯수입니다.
- `STRUCT_EX_PATH`: Execution Unit의 종류입니다. 만약 BRANCH유닛x1 ALUx3 LDST유닛x2이면 EX의 종류인 3으로 작성합니다.
- `STRUCT_RS_OUT_ENTRY[STRUCT_EX_PATH]`: EX_PATH에 따른 RS FIFO에서 출력되는 엔트리의 갯수입니다.
  각 EX_PATH에 따른 EX 유닛 수와 동일합니다.
- `STRUCT_EX_CORES`: Execution Unit의 총 개수 입니다. 만약 BRANCH유닛x1 ALUx3 LDST유닛x2이면 EX의 개수인 6으로 작성합니다.
- `STRUCT_EX_OUT_RESULT[STRUCT_EX_CORES]`: EX_PATH에 따른 EX에서 출력되는 엔트리의 갯수입니다.
  대부분의 경우에서 각 EX_PATH에 따른 EX 유닛 수와 동일하지만,
  하나의 EX가 여러 레지스터 출력을 수신할 수 있기에 `STRUCT_RS_OUT_ENTRY[STRUCT_EX_PATH]`와는 분리된 속성입니다.
- `STRUCT_PRM_ENTRY_UPDATE`: Physical Register Mapper에서 상태 변화가 필요한 IST 번호를 전달할때,
  한 사이클에 전송될 (IST 번호, 레지스터 번호) 쌍의 최대 개수입니다.
- `STRUCT_PRM_ENTRY_BUFFER`: Physical Register Mapper에서 하나의 내부 레지스터 번호에 연결된 IST 번호들이 저장되는 갯수입니다.
  만약 하나의 내부 레지스터 번호에 대해 이 이상 저장된다면, PRM 업데이트가 불가능하여 수신 불가 상태로 대기합니다.
  이후 해당 내부 레지스터가 저장되면 수신되면 수신 가능 상태로 변경합니다.
- `STRUCT_UNALLOCATE_PHYREG`: Flow Control Logic이 Physical Register Mapper으로 한 사이클에 반환시킬 수 있는 내부 레지스터의 수입니다.
- `STRUCT_FLOW_WINDOWS`: 점프/분기로 명령 윈도우가 구분되는, Flow Index로 사용되는 윈도우 수입니다.
- `STRUCT_FLOW_PC_MAX_RANGE`: 명령 윈도우의 Program Counter 최대 범위 입니다.

내부적으로 사용하기 위해 파라미터로 자동 생성되는 상수들은 `_`로 시작합니다.

- `_STRUCT_EX_OUT_RESULT_ALL`: `STRUCT_EX_OUT_RESULT[STRUCT_EX_CORES]`의 모든 요소 합입니다.
  상수에서 **비트 너비와 위치에 대한 상수가 따로 존재**하고, 이들의 이름 구조는
- `_BITWIDTH_*`: 데이터 구조에서 사용되는 비트 너비입니다.
- `_STARTPOINT_*`: 여러개의 데이터 구조 결합에서, 데이터 구조의 시작 위치 비트 번호입니다.
- `_ENDPOINT_*`: 여러개의 데이터 구조 결합에서, 데이터 구조의 끝 위치 비트 번호입니다.
  - `_*_LOW_*`: 최소 단위 데이터 구조를 뜻하는 명칭입니다.
  - `_*_CMB_*n*`: 두개 이상의 단위를 결합한 데이터 구조를 뜻하는 명칭입니다.
    LSB부터 결합된 순서대로 작성하며, `n`로 구분합니다.
  - `_*_IO_*`: 입출력 단위로 구성된 데이터 구조를 뜻하는 명칭입니다.

앞에서 설명한 구조/파라미터를 이용한 상수이기에, 구체적인 *비트 너비와 위치에 대한 상수*들은 따로 기술하지 않습니다.

이들을 결합하여 `_BITWIDTH_LOW_IS_INST_IMM`, `_STARTPOINT_CMB_OPCODEnFUNC_OPCODE` 와 같은 형태로 작성합니다.

동작의 예시를 설명해주세요!

동작을 설명하기 위해 RV32I ISA를 사용하고
아래와 같은 구조를 사용한다고 가정하겠습니다.

- `STRUCT_DECODE_NEW_INST`: 1
- `STRUCT_EX_PATH`: 3 (Branch/ALU/LDST 각각 하나씩)
- `STRUCT_RS_OUT_ENTRY[STRUCT_EX_PATH]`: 1, 1, 1 (Branch/ALU/LDST 각각 하나씩)
- `STRUCT_EX_CORES`: 3 (Branch/ALU/LDST 각각 하나씩)
- `STRUCT_EX_OUT_RESULT[STRUCT_EX_CORES]`: 1, 1, 1 (Branch/ALU/LDST 각각 하나씩)
- `STRUCT_PRM_ENTRY_UPDATE`: 3
- `STRUCT_PRM_ENTRY_BUFFER`: 4
- `STRUCT_UNALLOCATE_PHYREG`: 4
- `STRUCT_FLOW_WINDOWS`: 8
- `STRUCT_FLOW_PC_MAX_RANGE`: 8

~~구체적인 이해를 위해 위 구조로 체험형 플래어크라운드 웹앱을 추가로 제공합니다.~~
~~플래어크라운드 웹앱 링크~~

(아직 없습니다)

입력되는 예시 RV32I 명령은

```
ADDI x2, zero, 10
ADDI x3, zero, 7
ADDI x5, zero, zero
loop:
    ADDI x4, x5, 3
    ADD  x5, x3, x4
    BEQ  x2, x5, loop
```

이며, 을숙도 아키텍처 구조 자체는 LDST는 ALU와 동일한 구조를 가지므로 Branch와 ALU의 동작으로만 보여드립니다.
또한 내부 모듈들이 처리하는 자세한 내용은 각 모듈별 설명페이지에 동일한 구조와 명령으로 설명합니다.

PC:0, ADDI x2, zero, 10

1. 명령 요청 및 해석 [FCL, NEL]

- **[FCL]** Instruction Memory에 PC: 0번지 명령을 요청합니다.
  현재 분기 명령을 만나지 않았으므로 명령 윈도우인 **Flow Index는 0**입니다.
- **[NEL]** 새로운 명령 `ADDI`를 수신하고 해석합니다.
- 목적지 레지스터 `x2`를 위해 PRM에 새로운 물리 레지스터 번호 할당을 요청합니다.
  PRM의 `allocator_start_one`에 의해 비어있는 물리 레지스터 `P1`이 할당됩니다.
- 소스 레지스터 `zero`(x0)는 값이 0으로 고정된 레지스터이므로
  별도의 물리 레지스터를 할당받지 않고 항상 '준비 완료(Ready)' 상태로 취급됩니다.

2. 상태 저장 및 발급 대기 [IST, PRM]

- **[IST]** 내부 명령 체계(Micro-Op)로 변환된 이 명령은 IST의 비어있는 엔트리(예: `I0`)에 저장됩니다.
- 필요한 소스 레지스터(`zero`)가 이미 준비된 상태이므로,
  이 명령은 IST에 들어가자마자 실행 가능(Ready) 상태가 됩니다.

3. 실행 유닛으로 전달 [RS]

- **[RS]** IST에서 준비가 완료된 `I0` 명령을 받아 RS 내부의 ALU 큐(FIFO)로 전달합니다.
- ALU는 0 + 10 연산을 수행하여 결과값 10을 도출합니다.

4. 결과 브로드캐스트 및 상태 업데이트 [WBC, PRM]

- **[WBC]** 처리가 완료된 레지스터 번호 `P1`을 프로세서 내부(PRM, NEL, FCL)로 전달합니다.
- **[PRM]** `P1`을 기다리는 후속 명령의 IST 번호가 기록되어 있는지 확인합니다.
  대기 중인 명령이 없으므로 상태 변화 없이 대기 상태를 유지합니다.

5. 완료 [FCL]

- **[FCL]** `PC:0` 명령이 완료되었음을 확인합니다.

PC가 4, 8일 때는 위 구조와 동일하고 할당되는 레지스터만 달라집니다.
`PC:4`에서 x3는 물리 레지스터 `P2`를 할당받고 값 7을 가짐. `PC:8`에서 x5는 물리 레지스터 `P3`를 할당받고 값 0을 가짐.

PC:12, ADDI x4, x5, 3

- **[NEL -> PRM]** 목적지 레지스터 `x4`를 위해 물리 레지스터 `P4`를 할당받습니다.
  소스 레지스터 `x5`는 현재 물리 레지스터 `P3`에 매핑되어 있음을 확인합니다.
- **[IST]** 이 명령은 IST 엔트리 `I3`에 저장됩니다. 소스 레지스터 `P3` 연산이 완료되지 않은 경우,
  명령은 대기(Wait) 상태로 지정됩니다.
- **[PRM]** PRM은 `P3` 레지스터 주소에 대기 중인 IST 엔트리 `I3`을 기록합니다.
- **[WBC -> PRM -> IST]** 앞선 `PC:8` 연산이 끝나 WBC가 `P3` 완료 신호를 전달하면,
  PRM은 이를 수신하여 기록된 `I3` 엔트리에 상태 변경 신호를 송신합니다.
  `I3`는 실행 가능(Ready) 상태로 변경되어 RS로 전달됩니다.

PC:16, ADD x5, x3, x4

- **[NEL -> PRM]** 목적지 레지스터 `x5`를 위해 PRM에서 새로운 물리 레지스터 `P5`를 할당받습니다.
  소스 레지스터 `x3`는 `P2`, `x4`는 `P4`로 각각 매핑합니다.
- **[IST]** 해당 명령은 IST 엔트리 `I4`에 저장됩니다.
  소스 레지스터인 `P2`와 `P4`의 완료 신호가 모두 확인되면 실행 가능 상태로 변경되어 RS로 발급됩니다.

PC:20, BEQ x2, x5, loop

- **[NEL]** 분기 명령인 `BEQ`를 수신합니다. 이때 분기 명령 여부를 `FCL`에 전달합니다.
  소스 레지스터 `x2`는 최초 할당된 물리 레지스터 `P1`을, `x5`는 직전 명령에서 할당된 `P5`를 참조합니다.

- **[FCL]** 분기 명령이 입력됨에 따라, 이후 유입되는 명령어들은 새로운 명령 윈도우인 **Flow Index 1**로 할당됩니다.
  또한 Branch EX 유닛이 이 명령을 완료할때 까지 새로운 명령을 요구하지 않습니다.
  Flow Index 0에서 할당된 레지스터 중에 새로운 레지스터로 할당되면서 덮어씌워진 기존 물리 레지스터가 존재할 경우, PRM으로 반환합니다.
- **[WBC -> FCL]** Branch EX 유닛이 `P1`과 `P5`를 비교 연산하여 분기 조건 참(Taken)을 판별하고, 대상 주소(`PC:12`)를 WBC로 출력합니다.
  FCL은 이를 수신하여 `PC:12`부터 명령을 다시 요청합니다.

두 번째 루프 진입 (PC:12)

1. 새로운 명령 윈도우 할당 [FCL]

- **[FCL]** 분기 결과에 따라 `PC:12`부터 유입되는 명령어들은 **Flow Index 1** 구간에서 처리됩니다.

2. PC:12, ADDI x4, x5, 3

- **[NEL -> PRM]** 목적지 레지스터 `x4`를 위해 PRM은 비어있는 새로운 물리 레지스터 `P6`를 할당합니다.
- 소스 레지스터 `x5`는 1회차 루프에서 마지막으로 맵핑된 물리 레지스터 `P5`를 참조합니다.

3. PC:16, ADD x5, x3, x4

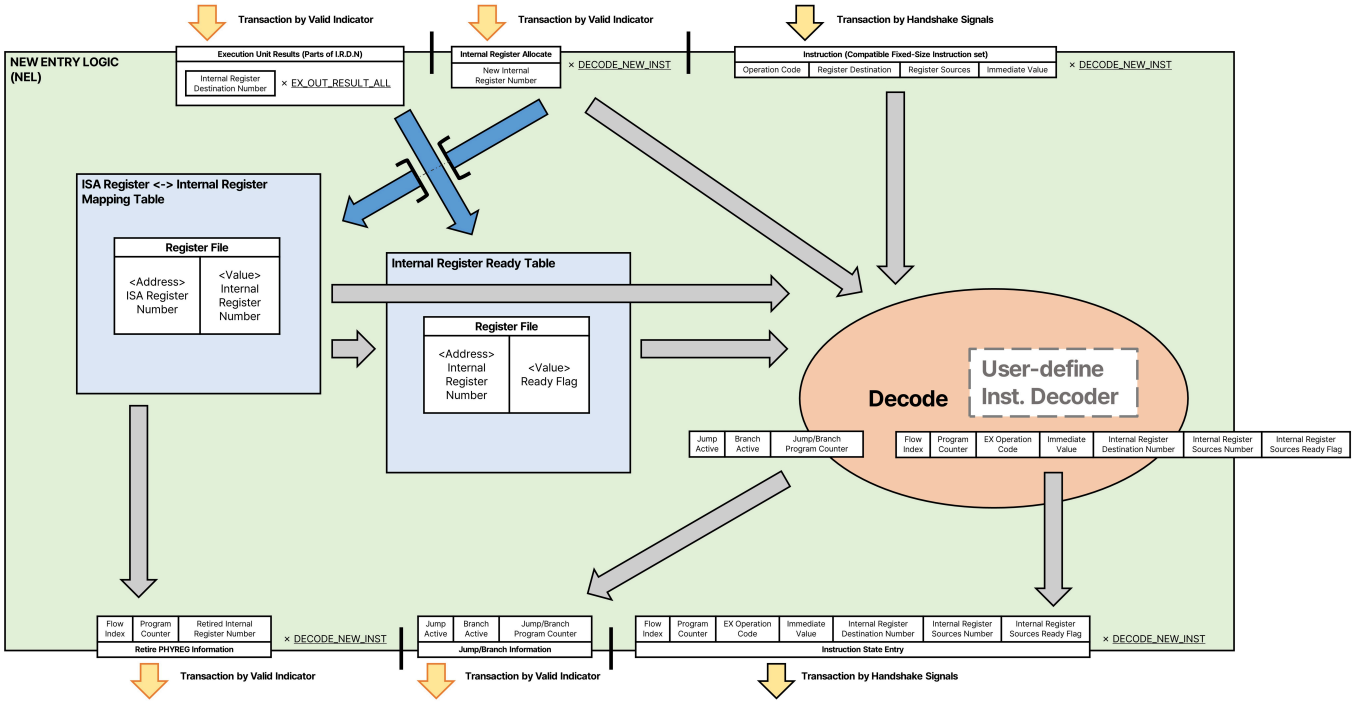
- **[NEL -> PRM]** 목적지 레지스터 `x5`를 위해 물리 레지스터 `P7`을 할당받습니다.
- 소스 레지스터 `x3`는 `P2`를 참조하고, `x4`는 2회차 진입 시 할당된 물리 레지스터 `P6`를 참조합니다.
  해당 명령은 `P6` 및 `P2`의 준비 상태에 따라 대기 및 발급 과정을 거칩니다.

4. PC:20, BEQ x2, x5, loop 및 자원 반환

- **[EX]** 2회차 루프의 `BEQ` 명령은 불변값인 `x2`(`P1`)과 2회차에서 갱신된 `x5`(`P7`)의 값을 비교 연산합니다.
- **[FCL]** Flow Index 0 (1회차 루프) 구간의 PC 내 모든 명령이 처리 완료되었음을 확인합니다.
- **[WBC -> FCL]** Branch EX 유닛이 `P1`과 `P5`를 비교 연산하여 분기 조건 참(Taken)을 판별하고, 대상 주소(`PC:12`)를 WBC로 출력합니다.
  FCL은 이를 수신하여 `PC:12`부터 명령을 다시 요청합니다.

New Entry Logic(NEL)

New Entry Logic는
새로운 명령을 입력받아, 이때 명령의 레지스터들을 내부 체계의 이름으로 변경하며,
새로운 명령에서 필요한 레지스터의 준비여부를 모아 내부에서 처리하는 명령 체계로 변환하는 모듈입니다.



내부의 구성과 역할

설계자 정의 ISA 디코더

을숙도 아키텍쳐는 설계자가 원하는 형태의 Instruction Set을 사용할 수 있습니다.
다만, 몇가지 조건이 있습니다.

- 고정 길이 명령 구조만 사용할 수 있습니다.
- 명령을 구분하는 PC 값의 단위가 반드시 동일하게 구분되어야 합니다.

설계자가 사용할 수 있는 디코더의 입출력 포맷이 고정되어 있습니다.

명령을 처리하기 위해 설계자가 정의 하는 ISA 디코더는
입력으로

Name	Bit-Width	Description
inst	IS_INST_BITWIDTH	Instruction Set 기반의 명령

을 받고,

출력으로

Name	Bit-Width	Quantity	Description
EX_PATH	BIT_WIDTH(STRUCT_EX_PATH)	1	EX 종류 지정
Micro-OP	EX_INST_MICROOP_BITWIDTH	1	EX용 Opcode
RD	BIT_WIDTH(IS_INST_REGS)	1	Register Destination
Allocate_NEW_RD	1	1	명령이 레지스터를 수정하는지 여부
RS	BIT_WIDTH(IS_INST_REGS)	IS_INST_OPERANDS	Register Sources
RS_use	1	IS_INST_OPERANDS	사용하는 Register Source 필드
Jump	1	1	PC의 값을 레지스터 없이 변경하는 명령 여부, 다음 명령을 불러오는 경우는 포함하지 않음
Jump_Reg	1	1	PC의 값을 레지스터를 이용하여 변경하는 명령 여부
Branch	1	1	PC의 값을 조건에 따라 변경하는 명령 여부

가 출력되게 만드셔야 합니다.
~~RV32I 디코더~~예제가 있습니다. 해당 구성으로 I/O를 만들면 됩니다.
이 디코더는 총 `STRUCT_DECODE_NEW_INST`개 만큼 존재하게 됩니다.

BIT_WIDTH(인자)는 인자의 log2에서 소수점 아래 값이 있을때 올림한 값입니다.

위와 같은 형태를 맞춰 설계자 정의 ISA 디코더를 설계한다면,
제공하는 생성기로 바로 을숙도 아키텍쳐에서 사용할 수 있습니다.

ISA Register <-> Physical Register Mapping Table

Instruction Set Architecture에 정의 된 레지스터와 내부 레지스터 간의 매핑 관계를 저장하는 Register File입니다.
Register File의 주소로 **Instruction Set 레지스터 번호**(너비: `_BITWIDTH_IS_INST_REGS`)를 사용하고,
Register File의 데이터로 **내부 레지스터 번호**(너비: `_BITWIDTH_STRUCT_PHYREGS`)를 저장합니다.

이 Register File 입출력 채널의 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST`
  - 아래부터 NEL 모듈에 입력된 순서로 전달하여
    NEL 모듈에 입력된 순서가 가장 낮은 명령 RD 레지스터 번호, ... , 입력되는 가장 높은 순서가 가장 높은 명령 RD 레지스터 번호 형태의 주소를 사용하고,
    동일한 순서로 **재할당된 레지스터 필드의 위치**(Write Enable)과 **매핑할 레지스터 값을 데이터로 입력**합니다.
- 출력 채널 갯수: `(STRUCT_DECODE_NEW_INST*(1+IS_INST_OPERANDS))`
  - 명령들의 RS_1 레지스터 번호, ... , 명령들의 RS_n 레지스터 번호, 명령들의 RD 레지스터 번호 묶음을
    아래부터 명령의 PC 값 순서로 전달하여
    [RS_1 (Inst `1`) / ... / (Inst `STRUCT_DECODE_NEW_INST`)], ~~~ ,
    [RS_`IS_INST_OPERANDS` (Inst `1`) / ... / (Inst `STRUCT_DECODE_NEW_INST`)],
    [RD (Inst `1`) / ... / (Inst `STRUCT_DECODE_NEW_INST`)]
    형태의 주소를 사용하고,
    동일한 순서로 **매핑 된 레지스터 값이 데이터로 출력**됩니다.

Physical Register Ready Table

내부 레지스터의 준비 여부를 저장하는 Register File입니다.
Register File의 주소로 **내부 레지스터 번호**(너비: `_BITWIDTH_STRUCT_PHYREGS`)를 사용하고,
Register File의 데이터로 **내부 레지스터의 준비 여부**(너비: 1)를 저장합니다.

이 Register File 입출력 채널의 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST+STRUCT_EX_CORES`
  - 디코더로 입력된 순서와 1번째 EX부터 마지막 EX 순서로
    *디코딩된 내부 RD 레지스터들의 번호와 EX에서 입력된 레지스터 번호들*의 형태의 주소를 사용하고,
    동일한 순서로 **초기화/업데이트 할 플래그 필드의 위치**(Write Enable)와 **디코딩 부분은 0, 업데이트 부분은 1을 입력**합니다.
    디코딩된 내부 RD 레지스터는 초기화용으로, EX에서 입력된 레지스터 번호들은 Ready 플래그 활성화 용으로 사용합니다.
- 출력 채널 갯수: `(STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS)`
  - 내부 레지스터로 변환된 명령의 RS_1 레지스터 번호, ... , 내부 레지스터로 변환된 명령의 RS_n 레지스터 번호 묶음을
    아래부터 명령의 PC 값 순서로 전달하여
    [(Inst `1`)PHY RS_1, ... , PHY RS_n] ~~~ [(Inst `STRUCT_DECODE_NEW_INST`)PHY RS_1, --- , PHY RS_n] 형태의 주소를 사용하고,
    동일한 순서로 **매핑 된 레지스터 값이 데이터로 출력**됩니다.

Decode

설계자 정의 ISA 디코더, ISA Register <-> Physical Register Mapping Table, Physical Register Ready Table에서 가져온 데이터로
내부 명령 체계로 변경하는 디코더입니다.

`STRUCT_DECODE_NEW_INST`가 1개라면 2단계로, `STRUCT_DECODE_NEW_INST`가 2개 이상이라면 3단계로 진행됩니다.

`STRUCT_DECODE_NEW_INST`가 한개라면

1. 입력된 명령을 해석(디코딩)하고, 필요하면 새 RD 레지스터를 할당하고,
   현재 상태의 RS 레지스터를 매핑된 내부 레지스터로 변환합니다.
2. 명령을 내부 명령체계로 변환하고,
   Physical Register Ready Table과 현재 입력된 EX 완료 레지스터까지 감안하여 Ready를 출력합니다.

`STRUCT_DECODE_NEW_INST`가 복수라면

입력된 명령간의 종속성을 확인하는 단계가 추가됩니다.

1. 입력된 명령을 해석(디코딩)합니다. 필요하면 새 RD 레지스터를 할당하고, 입력된 명령간의 종속성을 확인합니다.
2. 현재 상태의 RS 레지스터를 매핑된 내부 레지스터로 변환합니다. 이때, 입력된 앞쪽 명령의 RD와 뒤쪽 명령의 RS가 연결된 종속성이 존재하는 상황이라면, 종속성을 고려하여 변환합니다.
3. 명령을 내부 명령체계로 변환하고,
   Physical Register Ready Table과 현재 입력된 EX 완료 레지스터까지 감안하여 Ready를 출력합니다.

종속성 확인을 위해 **병렬 Suffix AND** 회로를 사용합니다.
규모에 따라 파이프라인 스테이지가 2사이클 이상의 사이클로 변경될 수 있습니다.

수신/송신하는 정보

정보 수집 I/O

NEL은 ISA 명령을 내부 명령 체계로 변경하기 위해 여러 부분에 정보를 수집합니다.

사용 가능한 내부 레지스터 번호를 PRM에서 수신

새로운 명령에서 레지스터 쓰기가 발생한다면, *레지스터 리네이밍*을 위해 새로운 레지스터를 할당받습니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

New Physical Register Number
[_BITWIDTH_STRUCT_PHYREGS-1:0]

이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 수신할 수 있습니다.

Handshake 기반 전송을 사용합니다.
다른 Handshake 기반 전송과 달리, Get 발생에 조건이 있습니다.

1. 디코딩된 명령이 새로운 레지스터를 요구해야 합니다.
2. 수신되는 Valid가 우선적으로 활성화되어야 하고, 모든 필드가 활성화 되어 있어야 합니다.
3. "새로운 명령을 Instruction Memory에서 수신"하는 부분의 모든 Valid 필드가 활성화 되어야 합니다.
4. "내부 체계로 변경된 명령 코드를 IST로 전달"하는 부분에서 모든 Get 필드가 활성화 되어야 합니다.

이는 외부에서 입력되는 명령의 전달 순서를 보장하기 위해 사용됩니다.
(순차성이 존재하는 명령 간의 연결을 위해 필수적으로 사용되는 조건들입니다.)

배포용 소스 코드에서 명칭은 `i/o_prm_allocate_*` 입니다.

준비가 완료된 내부 레지스터 번호를 EX에서 수신

EX에서 처리가 완료된 내부 레지스터에 대해 Ready Flag를 관리하기 위해, 완료된 내부 레지스터 번호를 입력받습니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

Ready Physical Register Number
[_BITWIDTH_STRUCT_PHYREGS-1:0]

이 정보는 동시에 STRUCT_EX_CORES 만큼 수신할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_ex_done_reg_*` 입니다.

새로운 명령을 내부 명령 체계로 변환

NEL은 ISA 명령을 내부 명령 체계로 변경하는 로직을 가지고 있고, 이를 전달하는 인터페이스가 있습니다.

새로운 명령을 Instruction Memory에서 수신

새로운 ISA 기반의 명령을 입력받습니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

New ISA Instruction
[IS_INST_BITWIDTH-1:0]

이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 수신할 수 있습니다.

Handshake 기반 전송을 사용합니다.
이 Handshake 신호가 NEL 모듈의 파이프라이닝 시작 신호와 동일합니다.
또한, **"내부 체계로 변경된 명령 코드를 IST로 전달"의 Handshake 부분에서 모든 Get 신호가 활성화 되어 있지 않는다면, 모든 필드에서 Get이 발생하지 않습니다.** 배포용 소스 코드에서 명칭은 `i/o_im_inst_*` 입니다.

내부 체계로 변경된 명령 코드를 IST로 전달

내부 명령 체계로 변환된 명령를 출력합니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

...RS Ready List...	...RS(n~1) Addresses List...	RD Address	Imm Value	Micro-Op
[IS_INST_OPERANDS-1:0]	[(_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS)-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[IS_INST_IMM-1:0]	[EX_INST_MICROOP

이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 전달할 수 있습니다.

Handshake 기반 전송을 사용합니다.
모든 필드의 Get 신호가 발생하지 않는다면, 모든 필드에서 Valid 신호를 전달하지 않습니다. 배포용 소스 코드에서 명칭은 `i/o_ist_newinst_*` 입니다.

내부 반환 및 점프/분기 명령 정보를 전달

내부 레지스터 반환을 위해 덮어 씌워지는 내부 레지스터 번호를 전달하고, 명령 윈도우의 재설정을 위해 점프/분기 명령 정보를 내보냅니다.

덮어 씌워지는 내부 레지스터 목록을 FCL로 전달

추후 명령 윈도우가 모두 처리되었을때 사용되지 않는 내부 레지스터 반환을 위해
덮어 씌워지는 내부 레지스터 번호를 전달합니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

Retired Physical Register Number
[`_BITWIDTH_STRUCT_PHYREGS`-1:0]

이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 전달할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_fcl_unallo_reg_*` 입니다.

점프/분기 명령 정보를 FCL로 전달

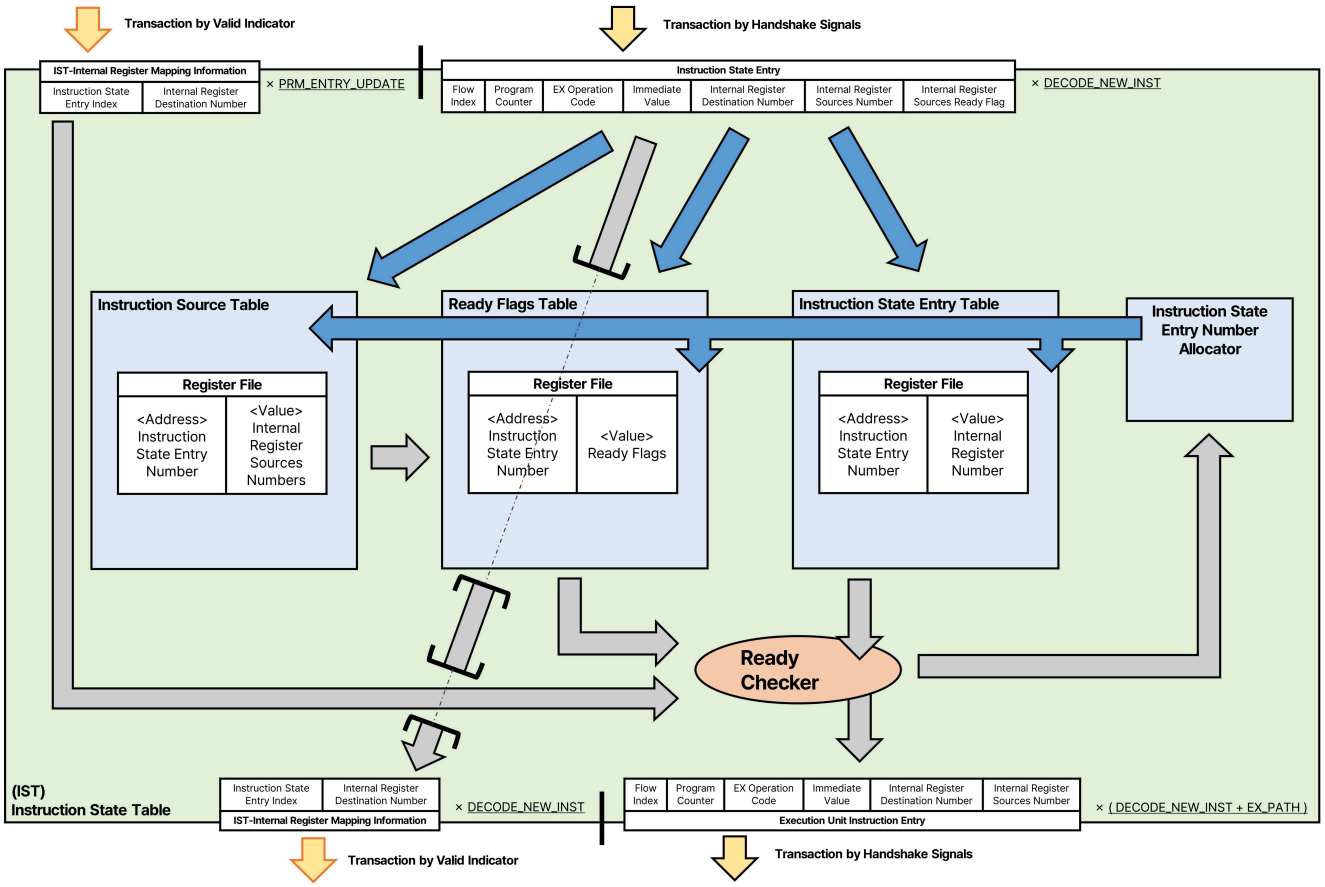
점프/분기가 발생하는 명령 정보를 전달합니다. 디코딩 되는 가장 앞선 점프/분기 명령에 대해서만 전달합니다. 이 정보는 하나만 전달할 수 있습니다.
Valid 기반 전송을 사용하는데, 다른 규격과 달리 주소와 플래그가 전달됩니다.

- jump[0]: Immediate 값을 이용한 점프 명령 여부
- jump_reg[0]: 레지스터 값을 이용한 점프 명령 여부
- branch[0]: 분기 명령 여부
- new_pc[`IS_INST_BITWIDTH`-1:0]: 점프/분기로 변경되거나 변경될 수 있는 PC. *단, jump_reg 발생에서는 사용하지 않음*

배포용 소스 코드에서 명칭은 `i/o_fcl_jump_branch_*` 입니다.

Instruction State Table(IST)

Instruction State Table은
입력된 명령에서, 준비된 명령은 처리할 수 있도록 전달하고,
아직 준비되지 않은 명령은 대기하는 모듈입니다.



내부의 구성과 역할

Instruction State Entry Number Allocator

Instruction State Table의 Entry를 할당하기 위한 Entry 번호 Allocator입니다.

이 Allocator는 Instruction State Table의 Entry 번호를 출력하고, 사용이 완료된 Entry 번호를 입력받습니다.
IST Entry 번호의 출력으로 `_BITWIDTH_STRUCT_INST_STATE_ENTRIES`만큼의 너비를 가지고,
이 정보는 동시에 STRUCT_DECODE_NEW_INST만큼 할당하고, 최대 STRUCT_PRM_ENTRY_UPDATE만큼 반환 할 수 있습니다.

Instruction Entry Table

대기하는 명령을 저장하는 Register File입니다.
Register File의 주소로 **Instruction State Entry 번호**(너비: `_BITWIDTH_STRUCT_INST_STATE_ENTRIES`)를 사용하고,
Register File의 데이터로 **내부 명령**(너비: `_BITWIDTH_INTERNAL_INST`)을 저장합니다.

`INTERNAL_INST`의 구조는

...RS(n~1) Addresses List...	RD Address	Imm Value	Micro-Op	EX Path
[(_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS)-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[IS_INST_IMM-1:0]	[EX_INST_MICROOP_BITWIDTH-1:0]	[_BITWID

와 같습니다. 내부 명령의 입력에서 Ready부분만 제외된 형태입니다.

이 Register File 입출력 채널 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST`
  - 아래부터 IST 모듈에 입력된 순서로 전달하여
    Instruction State Entry Number Allocator의 낮은 할당 필드 부터 높은 할당 필드에 출력된 Entry Number 형태의 주소를 사용하고,
    동일한 순서로 **입력되는 명령의 위치**(Write Enable)와 **입력되는 내부 명령을 데이터로 입력**합니다.
- 출력 채널 갯수: `STRUCT_PRM_ENTRY_UPDATE`
  - IST 엔트리 번호를
    RPM에서 수신된 필드 순서로 전달하여
    [IST Entry 1], ... , [IST Entry n]

형태의 주소를 사용하고,
동일한 순서로 **저장된 내부 명령이 데이터로 출력**됩니다.

Instruction Source Table

대기하는 명령에서 소스로 사용하는 내부 레지스터 번호를 저장하는 Register File입니다.
Register File의 주소로 **Instruction State Entry 번호**(너비: `_BITWIDTH_STRUCT_INST_STATE_ENTRIES`)를 사용하고,
Register File의 데이터로 **소스로 사용되는 내부 레지스터 번호**(너비: `_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS`)를 저장합니다.

이 Register File 입출력 채널 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST`
  - 아래부터 IST 모듈에 입력된 순서로 전달하여
    Instruction State Entry Number Allocator의 낮은 할당 필드 부터 높은 할당 필드에 출력된 Entry Number 형태의 주소를 사용하고,
    동일한 순서로 **입력되는 명령의 위치**(Write Enable)와 **입력되는 내부 명령을 데이터로 입력**합니다.
- 출력 채널 갯수: `STRUCT_PRM_ENTRY_UPDATE`
  - IST 엔트리 번호를
    RPM에서 수신된 필드 순서로 전달하여
    [IST Entry 1], ... , [IST Entry n]
    형태의 주소를 사용하고,
    동일한 순서로 **저장된 내부 명령의 소스 레지스터 번호들이 데이터로 출력**됩니다.

Ready Flags Table

대기하는 명령에서 준비된 내부 레지스터의 Ready Flag를 저장하는 Register File입니다.
Register File의 주소로 **Instruction State Entry 번호**(너비: `_BITWIDTH_STRUCT_INST_STATE_ENTRIES`)를 사용하고,
Register File의 데이터로 **소스 레지스터 준비 상태**(너비: 1)를 저장합니다.

이 Register File 입출력 채널 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST`
  - 아래부터 IST 모듈에 입력된 순서로 전달하여
    Instruction State Entry Number Allocator의 낮은 할당 필드 부터 높은 할당 필드에 출력된 Entry Number 형태의 주소를 사용하고,
    동일한 순서로 **업데이트하는 Ready Flag의 위치**(Write Enable)와 **업데이트하는 Ready Flag를 데이터로 입력**합니다.
- 출력 채널 갯수: `STRUCT_PRM_ENTRY_UPDATE`
  - IST 엔트리 번호를
    RPM에서 수신된 필드 순서로 전달하여
    [IST Entry 1], ... , [IST Entry n]
    형태의 주소를 사용하고,
    동일한 순서로 **저장된 Ready Flag가 데이터로 출력**됩니다.

IST-PHYREG Mapping Information

입력되는 내부 명령 체계에서 소스 레지스터와 현재 IST 엔트리 번호를 그대로 전달하는 로직으로,
입력되는 내부 명령 체계의 Ready Flag의 반전을 Valid로 적용합니다.
즉, **Ready Flag가 활성화 되지 않은 경우만 전달하는 구조**입니다.

Ready Checker

PRM에서 수신한 IST 엔트리 번호와 내부 레지스터 번호를 이용하여 Ready Flags Table을 업데이트 하고,
업데이트 하는 IST 엔트리의 모든 Ready Flag가 활성화 되는 경우 해당 내부 명령을 RS로 전달하는 로직입니다.

수신/송신하는 정보

소스 레지스터 번호를 대기열에 추가하거나, 사용 가능 상태를 수신

IST는 대기가 필요한 내부 레지스터 번호를 IST 엔트리 번호와 함께 전달하거나,
준비된 내부 레지스터 번호를 IST 엔트리 번호와 함께 수신받아 실행 가능한 명령 상태로 변경합니다.

대기가 필요한 내부 레지스터 번호를 IST 엔트리 번호와 함께 PRM으로 전달

새로운 명령이 들어오면 내부 레지스터 번호에 해당하는 대기열에 명령의 IST 엔트리 번호를 저장하기 위해
내부 레지스터 번호와 명령에 할당된 IST 엔트리 번호를 내보냅니다.
이때, 데이터의 유효성은 명령의 Ready Flag에 따르며, Ready Flag가 활성화 되지 않는 경우에 Valid 신호를 출력합니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

Instruction State Entry Number	Physical Register Number
[`_BITWIDTH_STRUCT_INST_STATE_ENTRIES`-1:0]	[`_BITWIDTH_STRUCT_PHYREGS`-1:0]

이 정보는 동시에 STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS 만큼 전달할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_prm_wait_phyreg_*` 입니다.

준비 완료된 내부 레지스터 번호와 IST 엔트리 번호와 함께 PRM에서 수신

준비가 완료된 내부 레지스터가 있다면, PRM에서 내부 레지스터 대기열에 있던 IST 엔트리를 입력받습니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

Instruction State Entry Number	Physical Register Number
[_BITWIDTH_STRUCT_INST_STATE_ENTRIES-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]

이 정보는 동시에 STRUCT_PRM_ENTRY_UPDATE 만큼 수신할 수 있습니다.

Handshake 기반 전송을 사용합니다.
"준비된 내부 명령을 RS로 전달"에서 모든 Get 신호가 활성화 되어 있지 않는다면, Get이 발생하지 않습니다.
배포용 소스 코드에서 명칭은 `i/o_prm_ready_phyreg_*` 입니다.

새로운 내부 명령을 수신하고, 준비된 내부 명령을 전달

새로운 내부 명령을 NEL에서 수신

새로운 명령을 NEL에서 입력받습니다.
이때 모든 Operand의 Ready Flag가 활성화 되어 있다면 통과하고, 아니라면 저장합니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

...RS Ready List...	...RS(n~1) Addresses List...	RD Address	Imm Value	Micro-Op
[IS_INST_OPERANDS-1:0]	[(_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS)-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[IS_INST_IMM-1:0]	[EX_INST_MICROOP

이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 수신할 수 있습니다.

Handshake 기반 전송을 사용합니다.
"준비된 내부 명령을 RS로 전달"에서 모든 Get 신호가 활성화 되어 있지 않는다면, Get이 발생하지 않습니다.
배포용 소스 코드에서 명칭은 `i/o_nel_newinst_*` 입니다.

준비된 내부 명령을 RS로 전달

준비된 명령을 RS로 내보냅니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

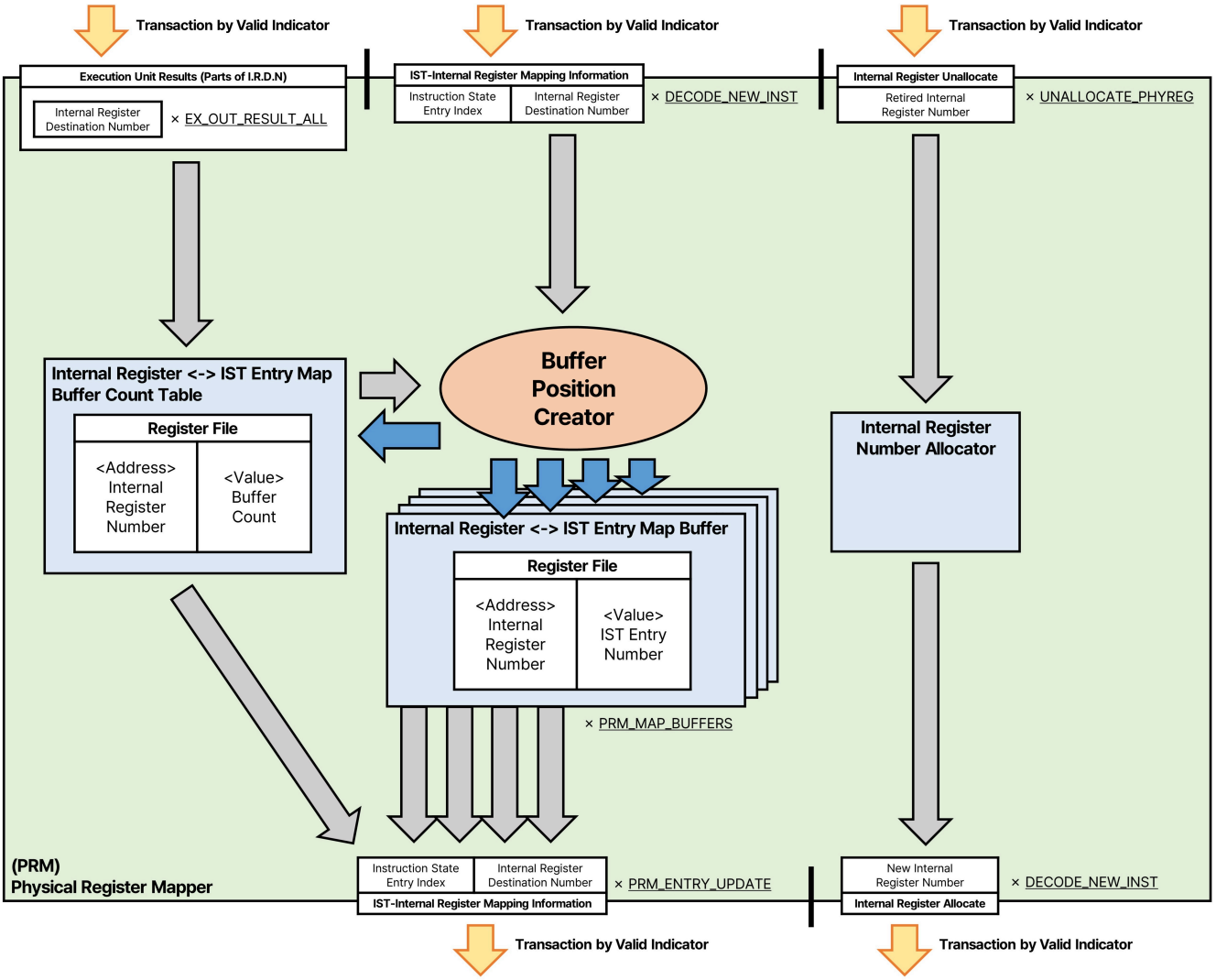
...RS(n~1) Addresses List...	RD Address	Imm Value	Micro-Op	EX Path
[(_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS)-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[IS_INST_IMM-1:0]	[EX_INST_MICROOP_BITWIDTH-1:0]	[_BITWID

이 정보는 동시에 STRUCT_DECODE_NEW_INST+STRUCT_PRM_ENTRY_UPDATE 만큼 전달할 수 있습니다.

Handshake 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_rs_readyinst_*` 입니다.

Physical Register Mapper(PRM)

Physical Register Mapper는
내부 레지스터 번호를 할당하고, 내부 레지스터에 연결된 명령 대기열을 관리하는 모듈입니다.



내부의 구성과 역할

Internal(Physical) Register Number Allocator

내부 레지스터를 명령에 사용할 수 있도록 할당하기 위한 내부 레지스터 번호 Allocator입니다.

이 Allocator는 내부 레지스터 번호를 출력하고, 더이상 사용되지 않는 내부 레지스터 번호를 입력받습니다.
내부 레지스터의 출력으로 `_BITWIDTH_STRUCT_PHYREGS`만큼의 너비를 가지고,
이 정보는 동시에 STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS만큼 할당하고, STRUCT_UNALLOCATE_PHYREG만큼 반환 할 수 있습니다.

Internal Register <-> IST Entry Map Buffer Count Table

내부 레지스터 번호에 대기중인 명령의 갯수를 저장하는 Register File입니다. Register File의 주소로 **내부 레지스터 번호**(너비: `_BITWIDTH_STRUCT_PHYREGS`)를 사용하고,
Register File의 데이터로 **대기열에 저장된 명령의 수**(너비: `_BITWIDTH_STRUCT_PRM_ENTRY_BUFFER`)를 저장합니다.

이 Register File 입출력 채널 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS`
  - 아래부터 IST 모듈에서 입력된 순서로 전달하여
    Operand에 사용된 내부 레지스터 주소를 입력된 명령과 Operand 순서로 묶인 형태의 주소를 사용하고,
    동일한 순서로 **업데이트 되는 명령의 Operand 위치**(Write Enable)와 **업데이트 이후 카운터 값을 데이터로 입력**합니다.
- 출력 채널 갯수: `(STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS)+_STRUCT_EX_OUT_RESULT_ALL`
  - 내부 레지스터 번호를
    IST의 Operand 묶음의 명령들, EX의 번호 순서대로 전달받아 주소를 사용하고,
    동일한 순서로 **저장된 카운터 값이 데이터로 출력**됩니다.

Internal Register <-> IST Entry Map Buffer

여러개가 배치되어 결과를 대기하는 내부 레지스터에 할당된 명령을 저장하는 대기열입니다.
총 `STRUCT_PRM_ENTRY_BUFFER`개의 Internal Register <-> IST Entry Map Buffer가 배치됩니다.
Register File의 주소로 **내부 레지스터 번호**(너비: `_BITWIDTH_STRUCT_PHYREGS`)를 사용하고,
Register File의 데이터로 **Instruction State Entry 번호**(너비: `_BITWIDTH_STRUCT_INST_STATE_ENTRIES`)를 저장합니다.

이 Register File 입출력 채널 구성으로

- 입력 채널 갯수: `STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS`
  - 아래부터 IST 모듈에서 입력된 순서로 전달하여
    Operand에 사용된 내부 레지스터 주소를 입력된 명령과 Operand 순서로 묶인 형태의 주소를 사용하고,
    동일한 순서로 **카운터로 추출된 업데이트 되는 위치와**(Write Enable)와 **Instruction State Entry 번호를 데이터로 입력**합니다.
- 출력 채널 갯수: `_STRUCT_EX_OUT_RESULT_ALL`
  - 내부 레지스터 번호를
    EX의 번호 순서대로 전달받아 주소를 사용하고,
    동일한 순서로 **Instruction State Entry 번호가 데이터로 출력**됩니다.

Buffer Position Creator

내부 레지스터에 대한 카운터를 업데이트 하고, 대기열의 위치를 지정하기 위한 로직입니다.
내부 레지스터 번호에 대한 Bitmap/Suffix OR/Prefix Sum 회로를 이용하여 입력에서 내부 레지스터가 겹치는 경우를 확인하고, Internal Register <-> IST Entry Map Buffer Count Table의 출력을 이용하여 새로운 카운터 값을 결정하는 구조를 가집니다.

Output FIFO

대기열에 저장된 IST 엔트리 번호가 전달되는 FIFO 입니다.
가변 입출력 FIFO를 사용합니다.

이 FIFO 입출력 채널 구성으로(가변 입출력이므로, 최대를 서술합니다.)

- 최대 입력 채널 갯수: `STRUCT_PRM_ENTRY_BUFFER`
  - 위치와 상관 없이 데이터가 전달될 수 있고, 유효한 데이터는 Valid와 함께 전달합니다.
- 최대 출력 채널 갯수: `STRUCT_PRM_ENTRY_UPDATE`
  - 입력되는 유효한 데이터들이 LSB부터 모아져 정렬되며, Valid 필드는 유효한 데이터 입력 갯수만큼 LSB부터 연달아 출력됩니다.

수신/송신하는 정보

새로운 레지스터를 할당하고 반환

할당 가능한 내부 레지스터 번호를 NEL로 전달

할당 가능한 내부 레지스터 번호를 출력합니다. Output FIFO의 출력입니다.

데이터는 내부 레지스터 번호이며,
이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 전달할 수 있습니다.

Handshake 기반 전송을 사용합니다.
가져가기 신호(Get)이 우선 발생하는 다른 Handshake 기반 전송과는 다르게
데이터 유효성 신호(Valid)가 먼저 발생해야 하는 전송구조입니다.
할당 가능한 레지스터가 존재하면 항상 Valid 신호를 출력합니다.
배포용 소스 코드에서 명칭은 `i/o_nel_allocate_*` 입니다.

반환되는 내부 레지스터 번호를 FCL에서 수신

더이상 사용되지 않는 내부 레지스터 번호를 입력받습니다.

데이터는 내부 레지스터 번호이며,
이 정보는 동시에 STRUCT_UNALLOCATE_PHYREG 만큼 수신할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_fcl_unallocate_*` 입니다.

내부 레지스터 번호에 연결된 명령 대기열에 레지스터를 추가

대기열에 추가할 IST 번호를 IST에서 수신

아직 준비되지 않은 내부 레지스터를 소스로 사용하여 대기열에 추가해야 하는 명령의 IST 엔트리 번호를 입력받습니다.

데이터는 IST 엔트리 번호이며,
이 정보는 동시에 STRUCT_DECODE_NEW_INST*IS_INST_OPERANDS 만큼 수신할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_ist_unallocate_*` 입니다.

완료된 내부 레지스터 번호에 연결된 대기열의 명령들을 전달

처리가 완료된 내부 레지스터 번호를 WBC에서 수신

준비된 내부 레지스터 번호를 WBC에서 입력받습니다.

데이터는 내부 레지스터 번호이며,
이 정보는 동시에 _STRUCT_EX_OUT_RESULT_ALL 만큼 수신할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_wbc_phyreg_*` 입니다.

준비된 내부 레지스터의 대기열에 저장된 IST 엔트리 번호를 IST로 전달

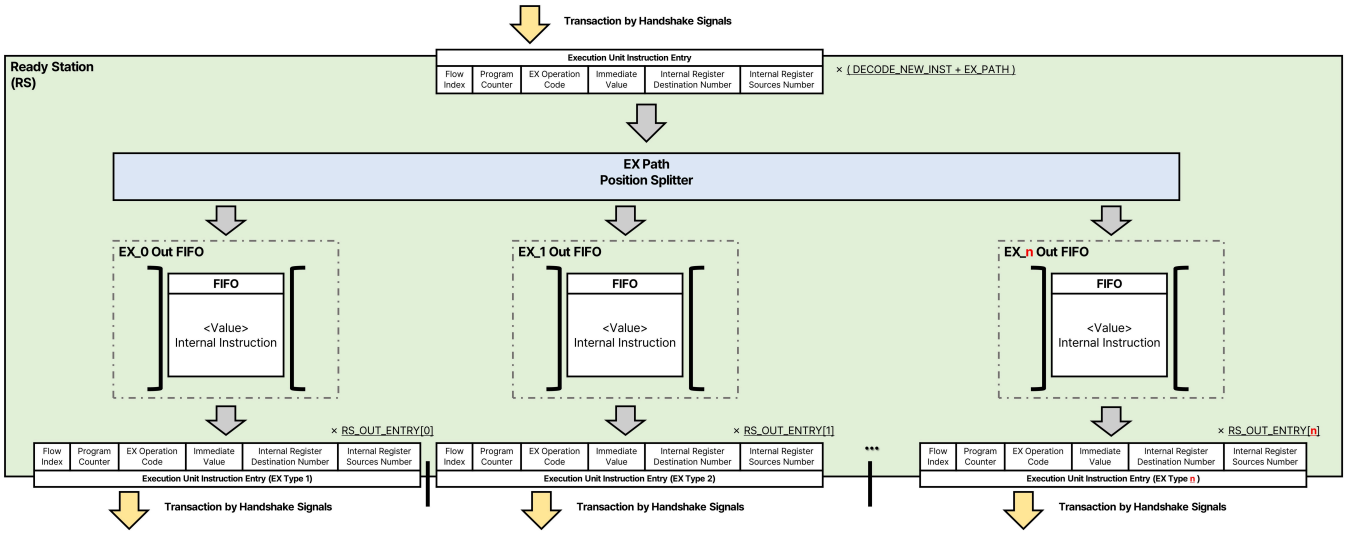
Internal Register <-> IST Entry Map Buffer에서 가져온 내부 레지스터 대기열의 IST 엔트리 번호들을 내보냅니다.

데이터는 IST 엔트리 번호들이며,
이 정보는 동시에 STRUCT_PRM_ENTRY_UPDATE 만큼 전송할 수 있습니다.

Handshake 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_ist_ready_phyreg_*` 입니다.

Ready Station(RS)

Ready Station은
준비된 명령이 EX에서 처리되도록
내부 명령이 대기하는 모듈입니다.



내부의 구성과 역할

EX Path Position Splitter

EX Path와 맞는 FIFO에 내부 명령을 전달하기 위한 Position Spliter 입니다.
내부적으로 **EX Path**로 할당된 Position Spliter가 있고, 입력에서 명령의 **EX Path** 필드와 비교하여
내부의 Position Spliter 번호와 동일한 경우 입력이 유효하도록 로직이 구성되어 있습니다.
(EX Path 번호와 내부의 Position Spliter 번호가 동일해야 내부의 Position Spliter의 Valid가 발생하도록 설계됩니다) Position Spliter의 특징에 따라 LSB 쪽으로 데이터가 정렬됩니다.

입력으로

Program Counter	Flow Index	EX Path	Micro-Opcode	Immediate Value	RD Address	...RS(1~n) Addresses List...	...RS(1~n) Values List...

를 데이터 구조로 사용하며, 동시에 STRUCT_DECODE_NEW_INST+STRUCT_PRM_ENTRY_UPDATE 만큼 입력받을 수 있습니다.

출력으로

Program Counter	Flow Index	Micro-Opcode	Immediate Value	RD Address	...RS(1~n) Addresses List...	...RS(1~n) Values List...

를 데이터 구조로 사용하며, 입력에서 EX Path만 제외됩니다. 동시에 STRUCT_RS_OUT_ENTRY[EX Path] 만큼 출력할 수 있습니다.

EX Path FIFO

특정한 EX Path로 향하는 명령이 버퍼링 되는 FIFO 입니다.
가변 입출력 FIFO를 사용하며,
FIFO의 데이터는 (LSB에서 MSB로)

Program Counter	Flow Index	Micro-Opcode	Immediate Value	RD Address	...RS(1~n) Addresses List...	...RS(1~n) Values List...

입니다.

이 FIFO 입출력 채널 구성으로(가변 입출력이므로, 최대를 서술합니다.)

- 최대 입력 채널 갯수: `STRUCT_DECODE_NEW_INST`
  - 위치와 상관 없이 데이터가 전달될 수 있고, 유효한 데이터는 Valid와 함께 전달합니다.
- 최대 출력 채널 갯수: `STRUCT_RS_OUT_ENTRY[FIFO의 EX_PATH]`
  - 입력되는 유효한 데이터들이 LSB부터 모아져 정렬되며, Valid 필드는 유효한 데이터 입력 갯수만큼 LSB부터 연달아 출력됩니다.

수신/송신하는 정보

준비가 완료된 명령을 실행하도록 버퍼링

RS는 준비가 완료된 명령을 IST에서 받고 FIFO에 저장하며, FIFO의 출력을 EX로 전달합니다.

준비가 완료된 명령을 IST에서 수신

준비된 명령을 IST에서 입력받습니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

...RS(n~1) Addresses List...	RD Address	Imm Value	Micro-Op	EX Path
[(_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS)-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[IS_INST_IMM-1:0]	[EX_INST_MICROOP_BITWIDTH-1:0]	[_BITWID

이 정보는 동시에 STRUCT_DECODE_NEW_INST+STRUCT_PRM_ENTRY_UPDATE 만큼 수신할 수 있습니다.

Handshake 기반 전송을 사용합니다. **RS내의 FIFO의 FULL의 반전을 Get 필드로 사용합니다.**
배포용 소스 코드에서 명칭은 `i/o_ist_readyinst_*` 입니다.

준비가 완료된 명령을 EX에 전달

준비된 명령을 EX로 내보냅니다.
순서는 EX Path가 낮은 순부터 높은 순으로 전달되며,
특히 EX Path의 번호에 맞게 STRUCT_RS_OUT_ENTRY[EX Path 번호] 필드를 사용하고,
사용되지 않는 영역은 비워두어야 합니다.
(STRUCT_RS_OUT_ENTRY[EX Path 번호]만큼 그대로 EX의 입력에 할당됩니다. 절대 모으지거나 위치를 변경하지 않습니다.)

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

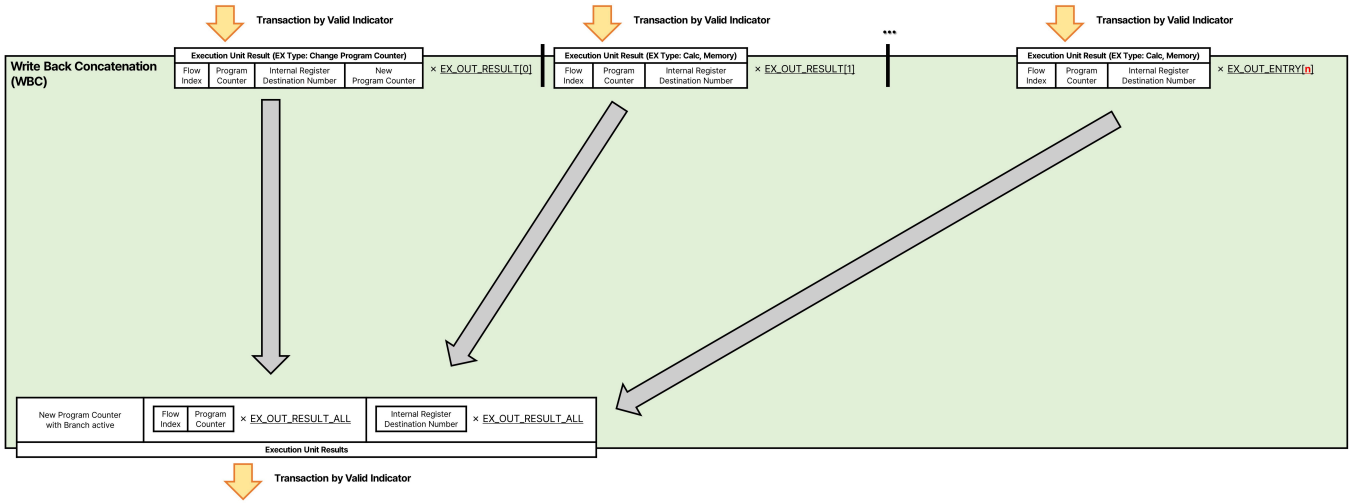
...RS(n~1) Addresses List...	RD Address	Imm Value	Micro-Op	Flow Ind
[(_BITWIDTH_STRUCT_PHYREGS*IS_INST_OPERANDS)-1:0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[IS_INST_IMM-1:0]	[EX_INST_MICROOP_BITWIDTH-1:0]	[_BITWID

이 정보는 동시에 STRUCT_RS_OUT_ENTRY[EX Path 번호]의 총합 만큼 전달할 수 있습니다.

Handshake 기반 전송을 사용합니다. **RS내의 FIFO의 EMPTY의 반전을 Get 필드로 사용합니다.**
배포용 소스 코드에서 명칭은 `i/o_ex_exeinst_*` 입니다.

Write Back Concatenation(WBC)

Write Back Concatenation은
EX에서 처리가 완료된 결과의 Flow Index, PC와
내부 레지스터 번호를 추출하여 전달하는 모듈입니다.



입력을 분할하여 바로 출력으로 전달하는 구조를 가지기 때문에 내부의 구성과 역할을 따로 서술하지 않습니다.
이 부분은 조합 논리로만 만들어집니다.

수신/송신하는 정보

처리가 완료된 결과의 Flow Index, PC, 내부 레지스터 번호를 추출해 전달

WBC는 EX에서 완료된 결과의 Flow Index, PC, 내부 레지스터 번호를 추출하고,
추출한 결과의 Flow Index, PC, 내부 레지스터 번호를 NEL, PRM, FCL에 전달합니다.

처리가 완료된 결과를 EX에서 수신

처리가 완료된 결과를 EX들에서 입력받습니다.
EX의 결과과 출력된 위치는 유효한 데이터 필드와 동일하며,
EX에 종류에 따라 데이터 필드의 구조는 바뀔 수 있습니다.

데이터 구조는 두가지이며, 첫번째는 분기 EX의 결과이고, 두번째는 일반적인 EX의 결과입니다.
MSB부터 LSB 순서로 아래와 같고,
(분기 EX의 결과)

New Program Counter	Branch Active	RD Address	Flow Index	Program Counter
[IS_INST_PC_BITWIDTH-1:0]	[0]	[_BITWIDTH_STRUCT_PHYREGS-1:0]	[_BITWIDTH_STRUCT_FLOW_WINDOWS-1:0]	[IS_INST_PC_BITWIDTH-1:0]

(일반적인 EX의 결과)

RD Address	Flow Index	Program Counter
[_BITWIDTH_STRUCT_PHYREGS-1:0]	[_BITWIDTH_STRUCT_FLOW_WINDOWS-1:0]	[IS_INST_PC_BITWIDTH-1:0]

이 정보는 동시에 _STRUCT_EX_OUT_RESULT_ALL 만큼 수신할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_ex_result_*` 입니다.

처리가 완료된 결과의 Flow Index, PC, 내부 레지스터 번호를 NEL, PRM, FCL으로 전달

입력 받은 결과에서 Flow Index, PC, 내부 레지스터 번호를 NEL, PRM, FCL으로 내보냅니다. 각각 내보냅니다.

Flow Index와 PC, 내부 레지스터 번호의 전송단위를 각각 나누어 전달합니다.
동일한 단위를 묶어 전달합니다. 단, 한쪽으로 모아서 전달하지 않습니다.
즉, 유효한 필드가 띄엄띄엄 존재할 수 있습니다.
(Flow Index와 PC)

Flow Index	Program Counter
[_BITWIDTH_STRUCT_FLOW_WINDOWS-1:0]	[IS_INST_PC_BITWIDTH-1:0]

(내부 레지스터 번호)

RD Address
[_BITWIDTH_STRUCT_PHYREGS-1:0]

이 정보는 동시에 _STRUCT_EX_OUT_RESULT_ALL 만큼 전달할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_(nel/prm/fcl)_(pc/phyreg)_*` 입니다.

처리가 완료된 Branch 결과를 FCL으로 전달

Branch EX에서 출력된 결과를 FCL로 내보냅니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

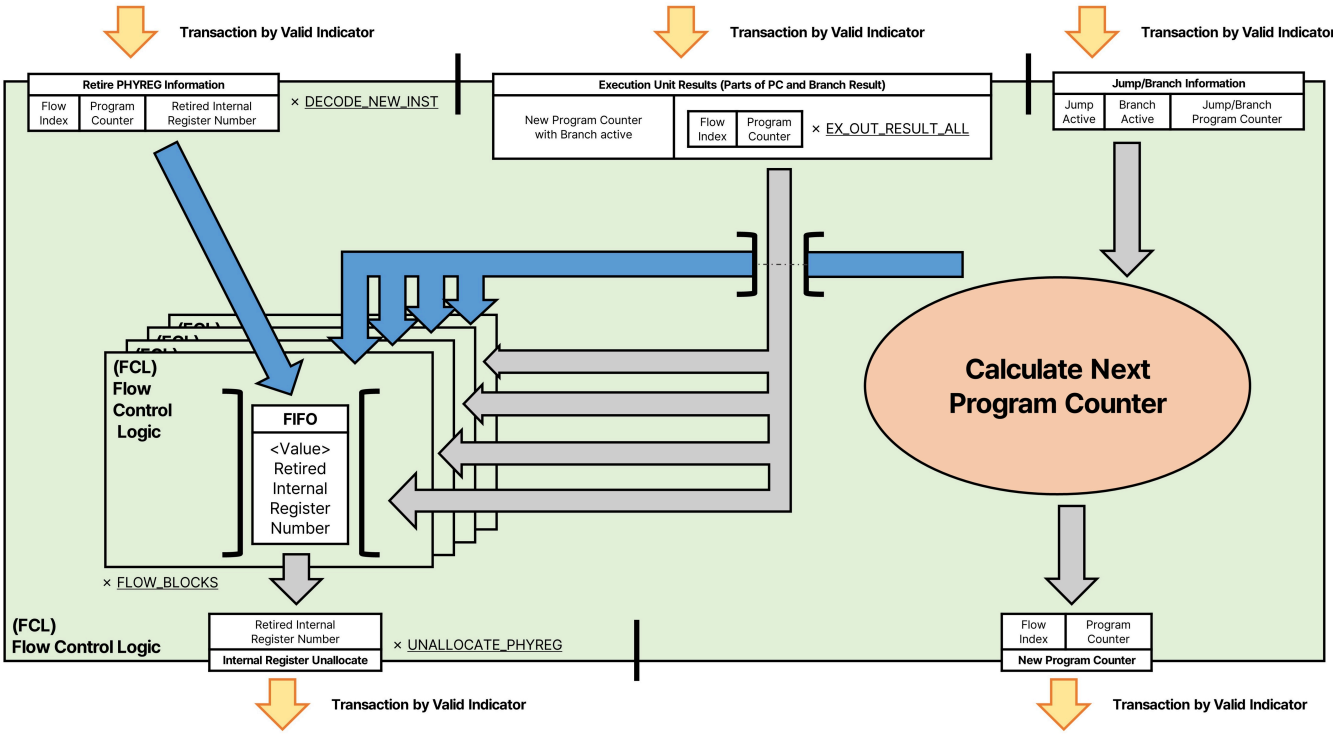
New Program Counter	Branch Active
[IS_INST_PC_BITWIDTH-1:0]	[0]

이 정보는 **오직 하나입니다.**

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_fcl_branch_*` 입니다.

Flow Control Logic(FCL)

Flow Control Logic은
명령의 흐름을 결정하기 위해 PC의 변화를 제어하고
덮어 씌워져 더이상 사용되지 않는 내부 레지스터 번호를 반환하는 모듈입니다.



내부의 구성과 역할

Flow Detect Unit(FDU)

명령 윈도우를 추적하고, 완료되면 더이상 사용되지 않는 내부 레지스터 번호를 반환하는 모듈이며,
내부에 윈도우 범위 비교기와 FIFO를 가지고 있습니다.
FCL 내부에 STRUCT_FLOW_WINDOWS 만큼 FDU가 있습니다.

명령 윈도우를 추적하기 위해 시작 PC, 종료 PC, 실행된 명령의 수, 명령 윈도우가 가지는 명령의 갯수를 저장합니다.
외부에서 종료 PC를 조작하여 명령 윈도우의 총 명령의 갯수를 업데이트하고,
입력받은 완료된 명령의 PC가 명령 윈도우 내에 해당하는 명령인지 확인하고, 이에 따라 실행된 명령의 수를 업데이트 합니다.

Calculate Next Program Counter

Program Counter를 조건에 따라 변경하고, 명령 윈도우를 생성하고 크기를 조정하는 로직입니다.

조건에 따라 PC를 업데이트 하는 방법이 달라집니다.

- *점프/분기 조건이 입력되지 않은 경우* 다음 명령의 PC로: 현재 명령 윈도우의 Flow Index와 현재 PC+`IS_INST_PC_STEP`를 전달합니다.
  - 단, 명령 윈도우의 상한(`STRUCT_FLOW_PC_MAX_RANGE`)까지 사용된 경우 명령 윈도우는 새롭게 설정되고, 새롭게 설정된 Flow Index를 전달합니다.
- *명령을 통해 즉시 점프가 가능한 명령이 입력된 경우* 다음 명령의 PC로: 새로운 명령 윈도우의 Flow Index와 점프되는 PC를 전달합니다. 이때 기존 명령 윈도우는 이전 명령까지 적용되도록 축소합니다.
- *레지스터 기반의 점프/분기 명령이 입력된 경우* 다음 명령의 PC로: 해당 명령이 완료되어 새로운 PC가 결정될 때까지 대기합니다. 이때 기존 명령 윈도우는 이전 명령까지 적용되도록 축소합니다.

수신/송신하는 정보

다음 명령의 PC를 전달

새로운 PC를 IM으로 전달

새로운 명령을 받기 위해 Instruction Memory(외부)에 새로운 명령의 Program Counter를 내보냅니다.

Calculate Next Program Counter에서 생성된 새로운 명령을 전달합니다.
PC는 하나만 전달되며, Instruction Memory는 전달된 PC를 시작으로 연달아 연결된 STRUCT_DECODE_NEW_INST개의 명령을 NEL로 전달해야 합니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_im_pc_*` 입니다.

점프/분기 명령 여부 수신

PC제어 명령의 정보를 NEL에서 수신

Program Counter를 변경하는 명령정보를 받고, 해당 명령의 종류에 따라 특정한 동작이 되도록
점프/분기/변경될 PC를 입력받습니다.
(Calculate Next Program Counter를 제어)

Valid 기반 전송을 사용하는데, 다른 규격과 달리 주소와 플래그가 수신됩니다.

- jump[0]: Immediate 값을 이용한 점프 명령 여부
- jump_reg[0]: 레지스터 값을 이용한 점프 명령 여부
- branch[0]: 분기 명령 여부
- new_pc[`IS_INST_BITWIDTH`-1:0]: 점프/분기로 변경되거나 변경될 수 있는 PC. *단, jump_reg 발생에서는 사용하지 않음* 딱 한세트만 전달되며,
  배포용 소스 코드에서 명칭은 `i/o_nel_jump_branch_*` 입니다.

완료된 명령들의 PC를 수신

실행이 완료된 명령들의 Flow Index와 Program Counter를 WBC에서 수신

명령 윈도우의 관리를 위해 실행이 완료된 명령의 Flow Index와 Program Counter를 입력받습니다.

실행이 완료된 명령 정보의 데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

Program Counter	Flow Index
[`_BITWIDTH_STRUCT_FLOW_WINDOWS`-1:0]	[`IS_INST_PC_BITWIDTH`-1:0]

이 정보는 동시에 _STRUCT_EX_OUT_RESULT_ALL 만큼 수신할 수 있습니다.
단, **첫번째 요소는 항상 분기 명령에 대한 요소**입니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_wbc_pc_*` 입니다.

처리가 완료된 Branch 결과를 WBC에서 수신

Branch EX에서 출력된 결과를 FCL로 내보냅니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

New Program Counter	Branch Active
[`IS_INST_PC_BITWIDTH`-1:0]	[0]

이 정보는 **오직 하나입니다.**

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_wbc_branch_*` 입니다.

덮어 씌워지는 내부 레지스터 번호를 수신

특정 명령 이후에 사용되지 않는 내부 레지스터 번호를 NEL에서 수신

추후 명령 윈도우가 모두 처리되었을때 사용되지 않는 내부 레지스터 반환을 위해
덮어 씌워지는 내부 레지스터 번호를 입력받습니다.

데이터 구조는 MSB부터 LSB 순서로 아래와 같고,

Retired Physical Register Number
[`_BITWIDTH_STRUCT_PHYREGS`-1:0]

이 정보는 동시에 STRUCT_DECODE_NEW_INST 만큼 수신할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_nel_unallo_reg_*` 입니다.

사용 완료된 내부 레지스터 번호를 반환

반환할 내부 레지스터 번호를 PRM에 전달

반환되는 내부 레지스터 번호를 FCL에서 수신

더이상 사용되지 않는 내부 레지스터 번호를 내보냅니다.

데이터는 내부 레지스터 번호이며,
이 정보는 동시에 STRUCT_UNALLOCATE_PHYREG 만큼 전달할 수 있습니다.

Valid 기반 전송을 사용합니다.
배포용 소스 코드에서 명칭은 `i/o_prm_unallocate_*` 입니다.